

NL DATA PROFILING Q&A

PROMPTS

1. Project Setup

Build an AI-powered data profiling assistant called NulAware AI.

Tech stack:

- Python
- Streamlit
- Gemini 2.5 Flash
- LangGraph
- ChromaDB
- SentenceTransformers
- SQLite
- Plotly
- ydata-profiling

The application should support:

1. CSV upload
2. Automatic profiling
3. Metadata extraction
4. Semantic chunking
5. Vector storage
6. RAG-based question answering
7. Interactive dashboard
8. PDF report generation
9. Conversation history

Provide a scalable folder structure.

2. CSV Upload Pipeline

Build a Streamlit upload module.

Requirements:

- Upload CSV files.
- Save files locally.
- Preview the first rows.
- Display column information.
- Show row count and column count.
- Add a button called "Profile and Index Dataset".

When clicked, trigger the profiling pipeline.

3. Profiling Module

Create a profiling module using ydata-profiling.

Requirements:

- Generate HTML report.
- Generate JSON report.
- Store both reports.
- Return report paths.

The code should be modular and production-ready.

4. Metadata Extraction

Build `extract_profile.py`.

Read the JSON generated by ydata-profiling and extract:

- Dataset summary
- Column statistics
- Missing values
- Correlations
- Outliers
- Duplicate rows

5. Chunking

Build a chunking module for RAG.

Convert metadata into chunks.

Examples:

Dataset summary chunk

Missing values chunk

Correlation chunk

Outlier chunk

Column statistics chunk

Each chunk should contain:

- text
- source

6. Embeddings

Create embeddings.py.

Use SentenceTransformer.

Requirements:

- Singleton model loading.
- Embed text chunks.
- Embed user queries.
- Return embeddings as lists of floats.

7. ChromaDB

Build a ChromaDB persistence layer.

Requirements:

- Store metadata chunks.
- Support multiple datasets.
- Retrieve top-k chunks.
- Persist collections on disk.

8. Retriever

Create retriever.py.

Input:

- question
- dataset name

Output:

Top relevant chunks with:

- source
- distance
- text

9. Visualization Module

Create Plotly visualization tools.

Support:

- Histogram
- Boxplot
- Heatmap
- Scatter plot
- Bar chart
- Pie chart
- Missing values chart
- Overview dashboard

Return Plotly figures.

10. PDF Report

Build report_tool.py.

Generate PDF reports containing:

- Executive summary
- Missing values

- Correlations
- Outliers
- Duplicate analysis
- Charts
- Recommendations

Return the PDF path.

11. Streamlit Interface

Create a modern Streamlit UI.

Tabs:

- Upload Dataset
- Chat With Dataset
- Visualization Dashboard
- Report Generator

Use custom CSS and dark theme styling.

12. Multi-Key Gemini

Implement Gemini API key rotation.

Requirements:

- Support multiple API keys.
- Round-robin mechanism.
- Retry on quota errors.
- Automatic failover.

13. Row-Level Analytics

This was one of the biggest upgrades.

Extend the system beyond metadata RAG.

Allow answering directly from dataframe rows.

Examples:

show ticket 230080

show passengerid 10

show cabin C85

show sex female

Build dataframe_tool.py containing:

find_column()

find_rows()

Integrate it with the LangGraph planner and graph.

14. Hybrid Agent

Transform NulAware AI from a metadata-only RAG system into a hybrid metadata + dataframe agent.

Metadata questions:

- Missing values
- Correlations
- Outliers
- Data quality

should use metadata.

Row-level questions should search directly inside the dataframe.

Route automatically through LangGraph.